

Aula 03 – Análise Assintótica de Algoritmos: Notação O e o

Norton T. Roman & Luciano A. Digiampietri
digiampietri@usp.br
@digiampietri

2023

Crescimento Assintótico de Funções

- O custo da solução aumenta com o tamanho n do problema
- O tamanho n fornece uma medida da dificuldade para resolver o problema
 - Tempo necessário para resolver o problema aumenta quando n cresce
- Exemplo:
 - Número de comparações para achar o maior elemento de um arranjo (array) ou para ordená-lo aumenta com o tamanho da entrada n .

Crescimento Assintótico de Funções

- A escolha do algoritmo não é um problema crítico quando n é pequeno
 - O problema é quando n cresce
- Por isso, é usual analisar o comportamento das funções de custo quando n é bastante grande
 - Analisa-se o comportamento assintótico das funções de custo
 - Representa o limite do comportamento do custo quando n cresce

Crescimento Assintótico de Funções

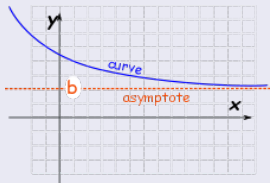
O que significa “Comportamento Assintótico”?

- O comportamento assintótico descreve uma função ou expressão com um limite ou assíntota definidos
 - A função pode se aproximar desse limite, na medida em que a entrada muda, mas nunca o alcançará
- Assíntota?
 - A linha da qual uma curva se aproxima enquanto caminha ao infinito

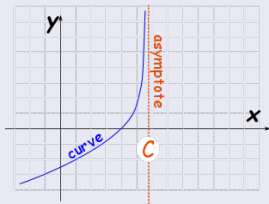
Comportamento Assintótico

Assíntotas

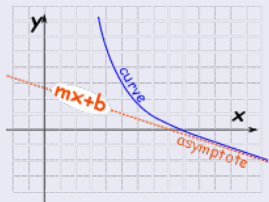
Horizontais



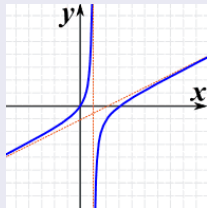
Verticais



Oblíquas



- Ex: $\frac{x-3x}{2x-2}$
 - Possui uma assíntota vertical em $x = 1$
 - E uma oblíqua em $y = \frac{x}{2} - 1$



Comportamento Assintótico

E o que isso tem a ver com complexidade?

- Seja $f(n)$ a função de complexidade de um algoritmo A
 - O comportamento assintótico de $f(n)$ representa o limite do comportamento do custo (complexidade) de A quando n cresce sem restrições
 - Lembrando que a função de complexidade geralmente considera apenas algumas operações elementares, ou mesmo uma única operação elementar (ex: o número de comparações)
- A complexidade assintótica relata o crescimento assintótico das operações consideradas

Comportamento Assintótico

E para que isso serve?

- Definir limites para o comportamento do algoritmo, identificando assim quando o barco vai afundar...
- Ex: 1 milhão (10^6) de operações por segundo

n						
Função de custo	10	20	30	40	50	60
n	0,00001s	0,00002s	0,00003s	0,00004s	0,00005s	0,00006s
n^2	0,0001s	0,0004s	0,0009s	0,0016s	0,0025s	0,0036s
n^3	0,001s	0,008s	0,027s	0,064s	0,125s	0,216s
n^5	0,1s	3,2s	24,3s	1,7min	5,2min	12,96min
2^n	0,001s	1,04s	17,9min	12,7dias	35,7 anos	366 séc.
3^n	0,059s	58min	6,5anos	3855séc.	10^8 séc.	10^{13} séc.

Comportamento Assintótico

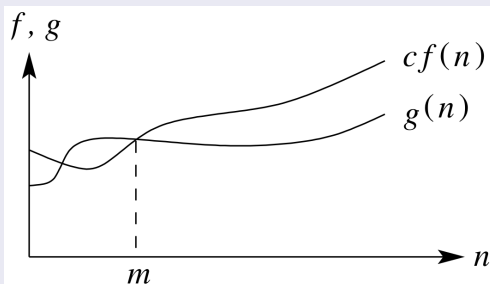
E para que isso serve?

- A definição de um limite nos dá uma caracterização simples da eficiência do algoritmo
- Mesmo podendo determinar sua complexidade exata, o cálculo dessa precisão extra pode não valer o esforço
- Para entradas grandes, as constantes multiplicativas e termos de menor ordem são dominados pelos efeitos do tamanho da entrada
- Nos permite também comparar o desempenho relativo de algoritmos alternativos
 - Nos diz qual será melhor quando a entrada cresce

Relacionamento Assintótico

Definição:

- Uma função $f(n)$ domina assintoticamente outra função $g(n)$ se existirem duas constantes positivas c e m tais que, para $n \geq m$, tem-se $|g(n)| \leq c \times |f(n)|$.



Relacionamento Assintótico

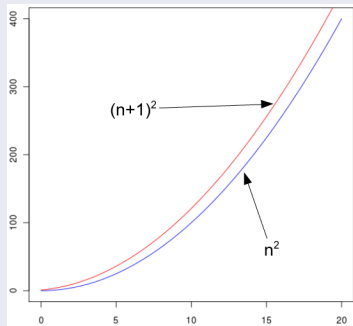
Quem domina quem?

- $g(n) = n$ e $f(n) = n^2$
 - $|n| \leq |n^2|$ para todo $n \in \mathbb{N}$
 - Para $c = 1$ e $m = 1$, temos que $|g(n)| \leq |f(n)|$. Portanto, $f(n)$ domina assintoticamente $g(n)$.
- $g(n) = n$ e $f(n) = -n^2$
 - $|n| \leq |-n^2|$ para todo $n \in \mathbb{N}$ (por ser módulo, o sinal não importa)
 - Para $c = 1$ e $m = 1$, temos que $|g(n)| \leq |f(n)|$. Portanto, $f(n)$ domina assintoticamente $g(n)$.

Relacionamento Assintótico

Quem domina quem?

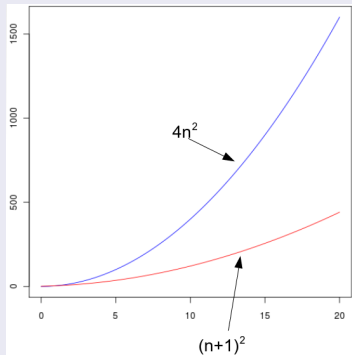
- $g(n) = (n + 1)^2$ e $f(n) = n^2$
 - Melhor por em um gráfico
 - $|n^2| \leq |(n + 1)^2|$ para $n \geq 0$
 - $g(n)$ domina $f(n)$
- Será somente isso?
 - Não há como $f(n)$ dominar $g(n)$?



Relacionamento Assintótico

Quem domina quem?

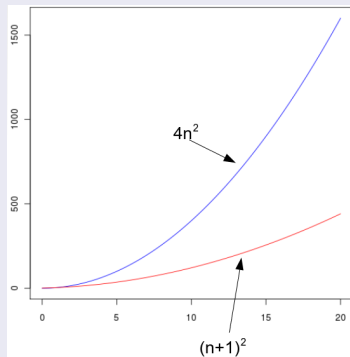
- Lembre que a definição envolve também uma constante
- Suponha que queremos $g(n) \leq cf(n)$
- Então $|(n+1)^2| \leq |cn^2|$
- Mas, para isso, basta que $|(n+1)^2| \leq |(\sqrt{c}n)^2|$ ou $|n+1| \leq |\sqrt{c}n|$
- Se $\sqrt{c} = 2$, ou seja, $c = 4$, isso é verdade, para $n \geq 1$



Relacionamento Assintótico

Quem domina quem?

- Então temos que $g(n)$ domina $f(n)$, pois $|n^2| \leq |(n+1)^2|$, $n \geq 0$
e $f(n)$ domina $g(n)$, pois $|(n+1)^2| \leq |4n^2|$, $n \geq 1$
- Nesse caso, dizemos que $f(n)$ e $g(n)$ dominam assintoticamente uma a outra



Notação O

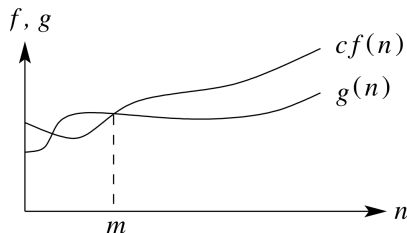
- Knuth (1968) criou a notação O (O grande) para expressar que $f(n)$ domina assintoticamente $g(n)$
- Escreve-se $g(n) = O(f(n))$ e lê-se: “ $g(n)$ é da ordem de no máximo $f(n)$ ”
- E para que serve isso?
 - Muitas vezes calcular a função de complexidade $g(n)$ de um algoritmo é complicado
 - É mais fácil determinar que $g(n)$ é $O(f(n))$, isto é, que assintoticamente $g(n)$ cresce no máximo como $f(n)$
 - Ex: Se dizemos que $T(n) = O(n^2)$, significa que existem constantes c e m tais que, para $n \geq m$, $T(n) \leq cn^2$

Notação O

Definição

Uma função $g(n)$ é $O(f(n))$ se existirem constantes positivas c e m tais que $0 \leq g(n) \leq cf(n)$, para todo $n \geq m$

- Informalmente, dizemos que, se $g(n) \in O(f(n))$, então $g(n)$ cresce no máximo tão rapidamente quanto $f(n)$
- Trata-se então de um **limite assintótico superior** para $g(n)$



Exemplo

- $\frac{3}{2}n^2 - 2n \in O(n^2)$? Faremos demonstrações completas na lousa, mas seguem resumos nos slides.
- Fazendo $c = \frac{3}{2}$ temos $\frac{3}{2}n^2 - 2n \leq \frac{3}{2}n^2$, para $m \geq 2$
- Outras constantes podem existir, mas o que importa é que exista alguma escolha para as constantes
- $(n + 1)^2 \in O(n^2)$?
 - Fazendo $c = 4$, $m = 1$, temos $(n + 1)^2 \leq 4n$
 - $(n + 1)^2 \in O(n^2)$ para $n \geq 1$

Exemplo

- $3n^3 + 2n^2 + n \in O(n^3)$?
 - Basta mostrar que $3n^3 + 2n^2 + n \leq 6n^3$, para $n \geq 0$ (ou seja, $c = 6$ e $m = 1$)
- $3n^3 + 2n^2 + n \in O(n^4)$?
 - Sim, mas essa afirmação é mais fraca que dizer que $3n^3 + 2n^2 + n$ é $O(n^3)$
 - Escolhemos então o limite “mais baixo”, pois nos interessa o assintoticamente mais próximo de $3n^3 + 2n^2 + n$
 - Isso, contudo, não implica estar errado que é $O(n^4)$

Notação O

Operações com a notação O

$$f(n) = O(f(n))$$

$$c \times O(f(n)) = O(f(n)) \text{ (} c: \text{ constante)}$$

$$O(f(n)) + O(f(n)) = O(f(n))$$

$$O(O(f(n))) = O(f(n))$$

$$O(f(n) + g(n)) = O(f(n)) + O(g(n))$$

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

$$O(f(n))O(g(n)) = O(f(n)g(n))$$

$$f(n)O(g(n)) = O(f(n)g(n))$$

Notação O

Operações com a notação O

- A regra $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$ pode ser usada para calcular a complexidade de uma sequência de trechos de um programa
- Ex: Suponha 3 trechos: $O(n)$, $O(n^2)$ e $O(n\log(n))$. Qual a complexidade do algoritmo como um todo?
 - Lembre-se que a complexidade total será a soma das complexidades de cada trecho
 - $O(n) + O(n^2) + O(n\log(n)) = \max(O(n), O(n^2), O(n\log(n))) = O(n^2)$

Notação O

Operações com a notação O

- Isso facilita muito o cálculo do limite superior para a complexidade de um algoritmo
- Ex: $3n^3 + 2n^2 + n \in O(n^3)$?
 - Sim, porque o termo de maior ordem ($3n^3$) é claramente $O(n^3)$
 - Então, pela regra, este será o limite da expressão como um todo
 - Não há necessidade de provar usando os termos de menor ordem

Notação O

Operações com a notação O

- Por vezes basta observar a estrutura do algoritmo para saber sua complexidade
- Como quando há um laço encadeado em outro, ambos proporcionais à entrada
- Nesse caso, essa parte é $O(n^2)$ e, se nenhuma outra for mais alta, então esse é o limite do algoritmo
- Mais do que isso, sendo O um limite superior, se o calcularmos no pior caso teremos um limite superior para toda e qualquer entrada

Problemas

- Como comparar algoritmos cujas complexidades são **equivalentes**?
 - Ou seja, quando $f(n)$ e $g(n)$ dominam assintoticamente uma à outra
- Nesses casos, o comportamento assintótico não serve para a comparação
 - Teremos que ver sua complexidade com mais detalhes, observando as constantes

Problemas

- Como comparar algoritmos quando não teremos entradas grandes?
 - Nesse caso, pode ocorrer que um algoritmo assintoticamente mais lento seja mais rápido para entradas pequenas
- Ex: um programa tem $f(n) = 100n$, e outro $g(n) = 2n^2$. Qual dos dois é melhor?
 - Depende do tamanho do problema
 - Para $n < 50$, o programa com tempo $2n^2$ é melhor do que o de $100n$
 - Então, se nunca teremos $n \geq 50$, o programa com $2n^2$ é melhor

Definição

Uma função $g(n)$ é $o(f(n))$ se, para toda constante $c > 0$, existe uma constante $m > 0$ tal que $0 \leq g(n) < cf(n)$, para todo $n \geq m$

- Informalmente, dizemos que, se $g(n) \in o(f(n))$, então $g(n)$ cresce mais lentamente que $f(n)$
- Intuitivamente, na notação o a função $g(n)$ tem crescimento muito menor que $f(n)$ quando n tende para o infinito
- Ou seja, $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$

Diferença entre O e o

- $O(f(n)) = \{g(n): \textbf{existem}$ constantes positivas c e m tais que $0 \leq g(n) \leq cf(n)$, para todo $n \geq m\}$
 - A expressão $0 \leq g(n) \leq cf(n)$ é válida para alguma constante $c > 0$
 - Basta acharmos um c e um m
- $o(f(n)) = \{g(n): \textbf{para toda}$ constante positiva c , existe uma constante $m > 0$ tal que $0 \leq g(n) < cf(n)$, para todo $n \geq m\}$.
 - A expressão $0 \leq g(n) < cf(n)$ é válida para toda constante $c > 0$
 - Para todo c temos que ter um m

Exemplo

- $1000n^2 \in o(n^3)$?
 - Buscamos um m tal que, para todo c e $n \geq m$,
 $1000n^2 < cn^3$
 - $\Rightarrow 1000 < cn$ (dividindo ambos os lados por n^2)
 - $\Rightarrow n > \frac{1000}{c}$
 - Ou seja, para todo valor de c , um m que satisfaz a definição é $m = \frac{1000}{c} + 1$ (pois $n \geq m$ e $n > \frac{1000}{c}$)

Exemplo

- $2n^2 \in o(n^2)$?
 - Buscamos um m tal que, para todo c e $n \geq m$, $2n^2 < cn^2$
 - Mas $2n^2 < cn^2 \Rightarrow c > 2$ (caso em que vale para todo $n > 0$)
 - Ou seja, não há m tal que, para todo c e $n \geq m$, $2n^2 < cn^2$
 - Logo, $2n^2 \notin o(n^2)$

Referências

- Ziviani, Nivio. Projeto de Algoritmos: com implementações em Java e C++. Cengage. 2007.
- Cormen, Thomas H., Leiserson, Charles E., Rivest, Ronald L., Stein, Clifford. Introduction to Algorithms. 2a ed. MIT Press, 2001.
- Gersting, Judith L. Fundamentos Matemáticos para a Ciência da Computação. 3a ed. LTC. 1993.
- <https://www.chegg.com/tutors/what-is-Asymptotic-and-Unbounded-Behavior/>
- <https://www.mathsisfun.com/algebra/asymptote.html>

Aula 03 – Análise Assintótica de Algoritmos: Notação O e o

Norton T. Roman & Luciano A. Digiampietri
digiampietri@usp.br
@digiampietri

2023